Studio IA Local · Manual técnico

1. Decisiones de diseño

El objetivo era un estudio **100% local, multimodal, multi-agente, con memoria persistente y workspace**, con el mejor equilibrio entre rendimiento y funcionalidad. Se han tomado como base los proyectos de referencia de cada área y se han integrado en una arquitectura ligera, sin depender de frameworks pesados:

Necesidad	Proyecto elegido	Por qué
Ejecutar LLM	**Ollama** (llama.cpp)	El estándar para inferencia local: GPU NVIDIA/AMD/Apple, cuantización GGUF, tool calling nativo, visión y embeddings en una sola API.
Modelos	**Qwen3**, **Qwen3-VL**, **Qwen3-Embedding**	Los mejores modelos abiertos en su tamaño para llamadas a herramientas, visión, código y español. Chat y programador comparten modelo para evitar cambios de modelo en la GPU.
Orquestación	Patrón **supervisor + ReAct** (como LangGraph Supervisor / CrewAI jerárquico)	Implementado en ~200 líneas: sin dependencias, trazable y rápido.
Memoria vectorial	**Qdrant**	Base vectorial de referencia; modo embebido en disco (sin servidor) o servidor en Docker.
Memoria del usuario	Enfoque **Mem0** (extracción + deduplicación) y **Letta/MemGPT** (memoria en capas)	Recuerdos útiles sin crecer sin control.
Voz a texto	**faster-whisper** (CTranslate2)	Hasta 4× más rápido que Whisper original, funciona en CPU int8 y GPU fp16.
Texto a voz	**Piper**	Voces neuronales muy rápidas en CPU, con voces en español.
Imagen	**ComfyUI**	El motor de difusión local más completo (SDXL, FLUX…), API HTTP.
Interfaz	Web propia en JavaScript sin dependencias (inspirada en Open WebUI)	Funciona sin Internet, carga al instante y sirve igual en PC, móvil y app Android.

2. Arquitectura

```
 Navegador / App Android ──HTTP+SSE──►  Backend FastAPI (Python)
                                        ├── agents.py   Supervisor ─► Investigador
/ Programador / Escritor / Artista
                                        │                └ bucle ReAct con
herramientas (tools.py)
                                        ├── memory.py   SQLite (conversaciones) +
Qdrant (hechos y documentos)
                                        ├── workspace.py carpeta compartida con
protección de rutas
                                        ├── multimodal.py faster-whisper · Piper ·
visión · ComfyUI
                                        └── llm.py ──► engines/ ──► Ollama · llama-
server · ONNX Runtime GenAI · OpenVINO GenAI
                                                                    LiteRT-LM ·
ExecuTorch · MLC LLM · TensorRT-LLM · Qualcomm Genie
```

Capa de motores (`backend/app/engines/`, v1.3) - `formats.py` reconoce el formato de cada modelo (nombre de Ollama, `.gguf`, carpeta ONNX GenAI con `genai_config.json`, OpenVINO IR, MLC, TensorRT-LLM, Genie con `genie_config.json`, `.pte`, `.litertlm`, Hugging Face). - `hardware.py` detecta CPU, RAM, GPUs con su VRAM (registro de Windows, `nvidia-smi`, `/proc`) y NPUs (Intel, AMD, Snapdragon). - `__init__.py` elige motor y ubicación por modelo: 1) lo fijado a mano, 2) lo medido con «⚡ Medir» (tokens/s por `motor|ubicación`), 3) una estimación según formato, tamaño y memoria. Los ajustes se guardan en `data/engines.json`. - Ubicación: `gpu` (VRAM), `cpu` (RAM), `mixed` (N capas en la GPU; Ollama `num_gpu`, llama-server `-ngl`) o `npu`. ONNX usa proveedores CUDA/DirectML/WebGPU/QNN; OpenVINO, los dispositivos CPU/GPU/NPU; LiteRT-LM, sus backends. - Los motores sin herramientas nativas reciben la plantilla Jinja del propio modelo (`tokenizer_config.json` o `chat_template.jinja`) y, si no admite herramientas, se describen en el prompt de sistema; las llamadas se extraen del texto (`<tool_call>`, `<|python_tag|>`, JSON…). - Los paquetes de Python de cada motor se instalan bajo demanda en `data/engines/site-packages` (botón «Instalar»). Embeddings de la memoria: siempre Ollama.

Flujo de un mensaje 1. Si hay imágenes, se guardan en `workspace/subidas` y el modelo de visión las describe. 2. Se recuperan de Qdrant los recuerdos relevantes y se añaden al contexto, junto a los 20 últimos mensajes. 3. El supervisor responde o llama a `delegate(agent, task)`. Cada especialista ejecuta su bucle ReAct (máx. `MAX_AGENT_STEPS`) con sus herramientas. 4. Cada paso se emite por Server-Sent Events: la interfaz muestra el proceso en directo. 5. Tras responder, el extractor de memoria destila hechos duraderos; si uno nuevo se parece >0,92 a otro existente, lo actualiza en lugar de duplicarlo.

Herramientas disponibles: `list_files`, `read_file` (texto, PDF, DOCX), `write_file` (indexa automáticamente), `search_workspace` (RAG), `remember`, `recall`, `run_python` (subproceso aislado con timeout, cwd en el workspace), `look_at_image`, `generate_image`.

3. Estructura del proyecto

```
backend/app/          API y lógica (config, llm, agents, tools, memory, workspace,
multimodal, main)
backend/tests/        prueba de extremo a extremo con un Ollama simulado
frontend/             interfaz web (index.html, app.js, style.css, markdown.js)
launcher/             lanzador StudioIA.exe en Go (Ollama, Python portátil, LAN, QR)
android/              app Android en Java + script de compilación sin Android Studio
build/                script del paquete de Windows
docker-compose.yml    despliegue en Linux/servidor (Ollama + Qdrant + estudio +
ComfyUI)
studio.env.example    toda la configuración
```

4. Configuración (`studio.env` o variables de entorno)

Variable	Defecto	Descripción
`MODEL_CHAT` / `MODEL_CODE` / `MODEL_VISION` / `MODEL_EMBED`	qwen3:8b / (= chat) / qwen3-vl:8b-instruct / qwen3-embedding:0.6b	Modelos de Ollama
`NUM_CTX`, `KEEP_ALIVE`, `THINK`, `TEMPERATURE`	8192, 30m, false, 0.6	Se envían en cada petición a Ollama (idénticas siempre, para que nunca recargue el modelo)
`WARMUP`	true	Precarga el modelo de chat y el de embeddings al arrancar
`TOOL_RESULT_CHARS`	8000	Máximo de cada resultado de herramienta que ve el modelo (`read_file` pagina con `offset`)
`OLLAMA_URL`	http://127.0.0.1:11434	
`QDRANT_URL`	(vacío = embebido en `data/qdrant`)	
`LAN`, `HOST`, `PORT`, `ACCESS_TOKEN`	false, 127.0.0.1, 8765	Acceso desde la red
`WHISPER_MODEL`, `WHISPER_LANGUAGE`, `WHISPER_BEAM`, `PIPER_VOICE`	small, (auto), 1, (auto en `voices/`)	Voz (GPU si hay CUDA; si faltan sus DLL pasa a CPU solo)
`COMFYUI_URL`		Generación de imágenes
`ENABLE_CODE_EXEC`, `CODE_EXEC_TIMEOUT`, `MAX_AGENT_STEPS`	true, 30, 8	Agentes
`ENGINE`, `PLACEMENT`, `GPU_LAYERS`	auto, auto, 0	Motor y ubicación por defecto (`gpu`, `cpu`, `mixed`, `npu`); cada modelo se ajusta en ⚙️ Modelos
`MODELS_DIR`, `ENGINE_PACKS_DIR`	`data/models`, `data/engines`	Modelos que no son de Ollama y motores instalados aparte
`LLAMA_SERVER`, `LLAMA_SERVER_URL`, `MLC_URL`, `TRTLLM_URL`, `GENIE_BIN`		Motores externos (ejecutable o servidor ya en marcha)

Variable	Defecto	Descripción
`MAX_NEW_TOKENS`	2048	Tokens máximos por respuesta fuera de Ollama

Optimizaciones de rendimiento (v1.2)

- **Streaming real**: el agente de primer nivel genera en streaming con herramientas (`/api/chat` con `stream: true`). Si el modelo acaba llamando a una herramienta, la interfaz recibe `reset`. Antes la respuesta se generaba entera y luego se simulaba el streaming.
- **Delegación directa** (`delegate` con `final: true`): si un solo especialista da la respuesta completa, se muestra tal cual en streaming y el supervisor no la reescribe. Ahorra una generación completa.
- **Caché de prefijo**: la memoria recuperada va en el mensaje del usuario, no en el prompt de sistema. El prefijo (sistema + historial) no cambia entre turnos y Ollama/llama.cpp reutilizan su caché KV.
- **Sin recargas**: `num_ctx` y `keep_alive` idénticos en todas las llamadas; chat y programador comparten modelo; `think` solo se envía a modelos que lo admiten (capacidades leídas de `/api/show` y cacheadas). Precarga al arrancar.
- **Contexto controlado**: historial con presupuesto de caracteres proporcional a `NUM_CTX` y resultados de herramientas recortados (`TOOL_RESULT_CHARS`), para que el modelo nunca trunque en silencio el prompt de sistema y las herramientas.
- **Memoria**: embeddings multilingües con el prefijo que espera cada modelo (consulta/documento), caché LRU, lotes de 32, troceado por párrafos y una colección de Qdrant por modelo de embeddings (cambiar de modelo no rompe las dimensiones). La extracción automática usa salida JSON estructurada, se salta mensajes triviales y se ejecuta **después** de liberar el chat (evento `done` antes de `memory_saved`). SQLite en modo WAL.
- **Herramientas**: llamadas escritas como texto (`<tool_call>{…}</tool_call>`) se rescatan; `read_file` pagina; lectura de PDF/DOCX y reindexado sin bloquear el servidor (hilos y 4 archivos en paralelo).
- **Voz**: faster-whisper con `beam_size=1`, idioma fijo opcional, hilos de CPU ajustados y paso automático a CPU si CUDA falla.
- **Interfaz**: Markdown repintado como mucho una vez por fotograma, botón ■ Parar (cancela la generación en Ollama), tiempo total y de primera palabra en cada respuesta, y en ⚙️ Modelos qué hay cargado y cuánto va en la GPU (`/api/ps`).
- **Ollama**: el lanzador fija `OLLAMA_FLASH_ATTENTION=1`, `OLLAMA_KV_CACHE_TYPE=q8_0`, `OLLAMA_KEEP_ALIVE=30m`, `OLLAMA_MAX_LOADED_MODELS=3` y `OLLAMA_NUM_PARALLEL=1`; si Ollama ya estaba abierto, las guarda como variables de usuario (`setx`) para el siguiente arranque.
- **Android autónomo**: llama-server con `--parallel 1 --flash-attn on --cache-type-k q8_0 --cache-type-v q8_0` (reintento automático sin ellos si el motor no arranca), servidor de embeddings con lotes de 2048 (antes fallaban textos de más de 512 tokens), historial que avanza a saltos de 4 mensajes para reutilizar la caché, y memoria automática solo cuando el mensaje cuenta algo personal, después de liberar el chat.

5. Ejecutar desde el código

```
cd backend
python -m venv .venv && . .venv/bin/activate   # Windows:
.venv\Scripts\activate
pip install -r requirements.txt
ollama pull qwen3:8b && ollama pull qwen3-vl:8b-instruct && ollama pull
qwen3-embedding:0.6b
python -m app.main            # http://127.0.0.1:8765
python tests/test_e2e.py      # prueba completa sin GPU (Ollama simulado)
```

Docker (Linux con o sin GPU NVIDIA)

```
docker compose up -d --build
docker compose exec ollama ollama pull qwen3:8b   # o desde la pestaña
Modelos
docker compose --profile image up -d              # + ComfyUI
(COMFYUI_URL=http://comfyui:8188)
```

6. Compilar los binarios

Windows (desde Linux o macOS; no requiere Windows):

```
pip install uv && go install github.com/tc-hib/go-winres@v0.3.3
bash build/build_windows.sh      # -> dist/StudioIA-Windows-x64.zip
```

Descarga Python 3.12 oficial (NuGet), instala las ruedas `win_amd64` con uv, compila el lanzador Go con icono y empaqueta.

Android (sin Android Studio):

```
bash android/build_android.sh    # -> dist/StudioIA-Android.apk y
dist/StudioIA-Android-Snapdragon.apk
```

Descarga apktool (aapt2 + framework), D8 (R8), apksig y la API de Android 14, y los motores: ONNX Runtime GenAI (NuGet), ONNX Runtime, ExecuTorch con Vulkan, fbjni y GenieX (Maven Central) y LiteRT-LM (rueda de PyPI para Android, más el puente `android/jni/litertlm_jni.c` compilado con zig). Compila recursos y Java, dexa, firma (v1 + v2) y verifica. `VARIANTS=standard` compila solo la normal. La clave se crea en `android/release.p12` (contraseña `studioia`, cambiable con `KEYSTORE_PASS`). **Guárdala**: las actualizaciones deben firmarse con la misma clave.

Modo autónomo de la app Android

- `android/build_engine.sh` compila `llama-server` (llama.cpp) para arm64 como binario estático con zig (sin NDK) y lo deja en `android/jniLibs/arm64-v8a/libllama-server.so`. `build_android.sh` lo empaqueta si existe.
- `LocalStudio.java` lanza el motor como proceso local (chat/visión en 127.0.0.1:8431, embeddings en :8432), sirve la interfaz desde los assets en `http://studio.local/` y expone el puente nativo (archivos, descargas, voz).

- `android/assets/www/local.js` sustituye al backend: intercepta `fetch('/api/…')` e implementa en el móvil agentes (supervisor + ReAct con herramientas por texto `<tool>{…}</tool>`, válido para cualquier modelo), memoria vectorial con deduplicación y workspace.
- Motores dentro de la app (`Engines.java`): ONNX Runtime GenAI (CPU), ExecuTorch (CPU con XNNPACK o GPU con Vulkan, según se exportó el `.pte`), LiteRT-LM (CPU o GPU) y, solo en la APK Snapdragon, Qualcomm Genie mediante GenieX (`src-snapdragon/`: NPU Hexagon, GPU Adreno o CPU; carpetas de Qualcomm AI Hub o `.gguf`). llama.cpp sigue como proceso aparte (CPU). `local.js` elige motor y ubicación por modelo igual que el PC (formato → motores posibles → lo fijado, lo medido o la estimación) y aplica la plantilla Jinja del modelo con `jinja.js` (@huggingface/jinja, MIT). OpenVINO, MLC y TensorRT no tienen versión utilizable en Android y aparecen con ✕ y el motivo.
- Prueba sin dispositivo: `node android/tests/local_test.js` (Chromium con puente y motores simulados).

7. API HTTP

Método y ruta	Uso
`GET /api/ping`, `GET /api/health`	Estado, modelos instalados/faltantes, capacidades
`POST /api/models/pull {model}`	Descarga un modelo (progreso NDJSON); `hf:org/repo` baja de Hugging Face
`GET /api/engines`, `PUT /api/engines/settings {models: {ref: {engine, placement, gpu_layers}}, roles}`	Motores, hardware y dónde se ejecuta cada modelo
`POST /api/engines/benchmark {model, max_tokens}`, `POST /api/engines/{id}/install`	Medir velocidad por motor y ubicación (NDJSON); instalar un motor
`POST /api/models/download {repo, include}`	Descargar un modelo de Hugging Face a `MODELS_DIR`
`POST /api/chat {message, images[], mode, conversation_id, auto_memory}`	Chat (SSE: `agent`, `tool_call`, `tool_result`, `memory`, `token`, `memory_saved`, `done`, `error`)
`GET/POST/PATCH/DELETE /api/conversations[/{id}]`	Conversaciones
`GET/PUT /api/agents`, `POST /api/agents/reset`, `GET /api/tools`	Equipo de agentes
`GET/POST /api/memory`, `GET /api/memory/search?q=`, `DELETE /api/memory/{id}`	Memoria
`GET /api/workspace`, `GET/PUT /api/workspace/file`, `GET /api/workspace/raw`, `POST /api/workspace/upload`, `DELETE /api/workspace`, `POST /api/workspace/reindex`	Workspace

Método y ruta	Uso
`POST /api/stt` (audio), `POST /api/tts {text}`, `POST /api/image {prompt}`	Multimodal

Con `ACCESS_TOKEN` definido, todas las rutas `/api` (salvo `ping`) exigen `Authorization: Bearer <token>`, `?token=` o la cookie `studio_token`.

8. Seguridad

- `run_python` ejecuta código con los permisos de tu usuario dentro del workspace. Desactívalo con `ENABLE_CODE_EXEC=false` si no lo necesitas.
- El workspace rechaza rutas fuera de su carpeta (`..`, absolutas).
- Por defecto el estudio solo escucha en `127.0.0.1`; `LAN=true` lo abre a la red local con token.

9. Ampliaciones sugeridas

- Búsqueda web local con SearXNG como herramienta adicional.
- Servidores MCP como herramientas de los agentes.
- Instalador MSI/Inno Setup firmado y publicación de la app en F-Droid.